

Primeira Entrega - Sistema “Seu Cantinho”

Heric Camargo GRR20203959 Mateus Ribamar GRR20190154

30 de outubro de 2025

0.1 1. Introdução

Este documento apresenta a primeira entrega do sistema “Seu Cantinho” para gerenciamento de reservas de espaços para festas e eventos, contemplando análise arquitetural e diagramas UML.

0.2 1.1 ASR’s (Requisitos Arquiteturalmente Significativos)

- **Consistência de Dados** (Alto): Evitar double-booking com transações confiáveis
- **Confiabilidade** (Alto): Robustez contra falhas e recuperação rápida
- **Usabilidade** (Médio): Interface simples considerando baixa familiaridade tecnológica
- **Escalabilidade** (Alto): Suporte para crescimento e múltiplas filiais
- **Manutenibilidade** (Alto): Facilidade para evolução e novas funcionalidades

0.2.1 1.2 Restrições Técnicas

- **REST**: API com documentação OpenAPI/Swagger
- **Docker**: Execução via `docker compose up`

0.3 2. Análise de Estilos Arquiteturais

Estilo	Adequação ao “Seu Cantinho”
Monolítico	Adequado para estágio inicial pela simplicidade e transações ACID nativas, mas pode limitar crescimento futuro para múltiplas filiais.

Estilo	Adequação ao “Seu Cantinho”
Microserviços	Inadequado: alta complexidade operacional injustificada para o porte do negócio, dificuldade em garantir consistência forte (double-booking) e requer expertise avançada.
Camadas	Escolha ideal: equilibra simplicidade operacional, manutenibilidade, consistência transacional e permite evolução gradual conforme crescimento do negócio.

0.3.1 2.1. Justificativa da Escolha: Arquitetura em Camadas

1. **Consistência crítica:** Transações ACID para prevenir double-booking
2. **Simplicidade:** Baixa complexidade operacional adequada ao contexto
3. **Custo-benefício:** Menor overhead de desenvolvimento e operação
4. **Evolução gradual:** Permite migração futura sem comprometer entrega inicial

0.4 3. Arquitetura em Camadas

Princípios: Separação de responsabilidades, dependência unidirecional (superior → inferior), alta coesão e baixo acoplamento, substituíbilidade de camadas.

Implementação correspondente:

- **src/app:** Express Router, controladores REST, middlewares (CORS, Helmet, error handler) e validadores Zod.
- **src/application:** Serviços especializados (**UserService**, **SpaceService**, **ReservationService**, **PaymentService**) que orquestram regras e verificações de disponibilidade.
- **src/domain:** Entidades modeladas como interfaces TypeScript, enums de status e **AppError** para padronizar exceções.
- **src/infra:** Repositórios **better-sqlite3** responsáveis por persistir dados no arquivo `data/database.sqlite`.

0.5 4. Diagramas UML

0.5.1 4.1. Diagrama de Classes

Entidades principais: **User**, **Space**, **Reservation** e **Payment**, alinhadas diretamente com as interfaces utilizadas na API. As reservas referenciam `userId` e `spaceId`, os pagamentos se ligam a uma reserva via `reservationId`, e os estados possíveis são representados por **ReservationStatus** (PENDING, CONFIRMED, CANCELLED) e **PaymentStatus** (SIGNAL, FULL).

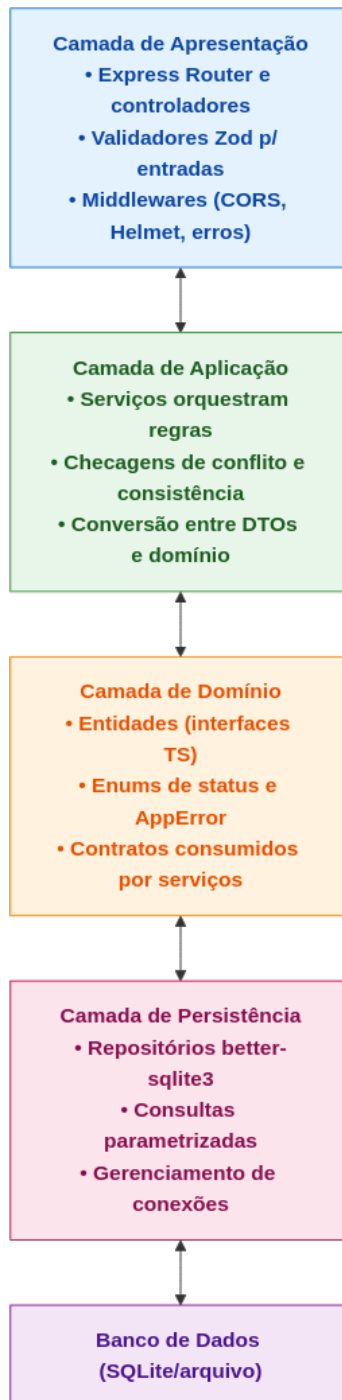


Figure 1: Estrutura Proposta

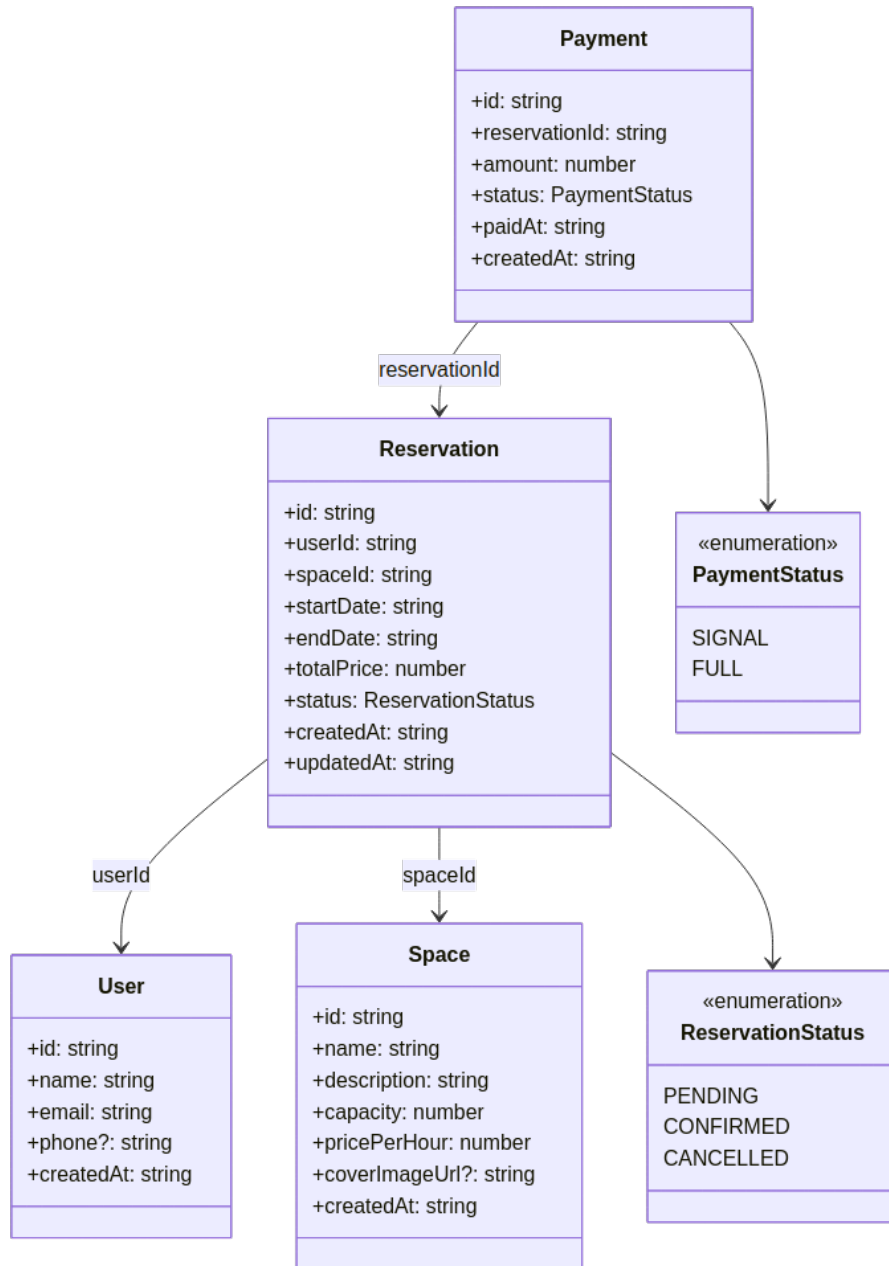


Figure 2: Diagrama de Classes

0.5.2 4.2. Diagrama de Componentes

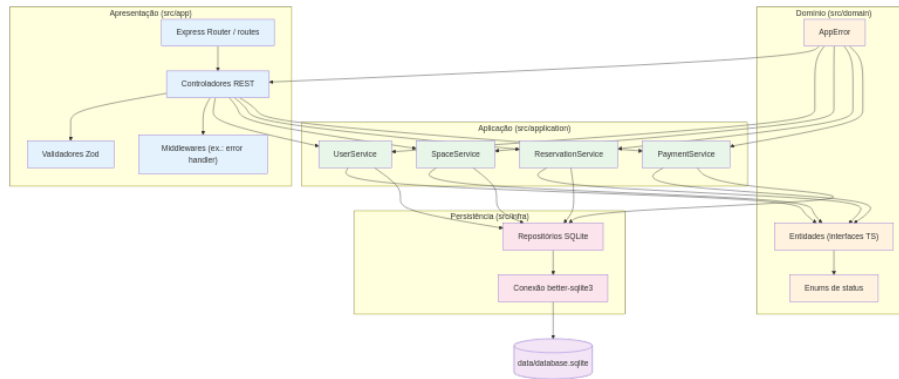


Figure 3: Diagrama de Componentes

Organização em 4 camadas: **Apresentação** (Express Router, controladores e validadores Zod), **Aplicação** (serviços por agregado), **Domínio** (interfaces, enums e erros) e **Persistência** (repositórios sobre `better-sqlite3` que gravam em um único arquivo SQLite montado por volume Docker).